

1. O que é encapsulamento?

O que significa encapsular dados em uma classe?

Encapsular dados em uma classe significa proteger os atributos do objeto, permitindo que eles sejam acessados ou modificados apenas por métodos controlados da própria classe. Dessa forma, a classe controla como seus dados são utilizados.

Qual é a diferença entre um atributo public, protected e private?

public: pode ser acessado de qualquer lugar do código.

protected: pode ser acessado apenas pela própria classe e por suas classes filhas (herança).

private: pode ser acessado somente pela própria classe.

Por que não é boa prática deixar todos os atributos como public?

Porque qualquer parte do programa poderia alterar os valores livremente, podendo gerar erros, inconsistências e dificultar a manutenção do código. O encapsulamento protege os dados e garante maior segurança e controle.

Exemplo:

Se um atributo idade for público, alguém poderia fazer:

```
$pessoa->idade = -10;
```

Isso criaria um valor inválido para a idade.

2. Para que servem os métodos get e set?

Por que usamos getAtributo() ao invés de acessar \$objeto->atributo diretamente?

Porque o método get permite controlar a forma como os dados são retornados, mantendo o encapsulamento da classe. Além disso, caso a lógica de acesso mude futuramente, não será necessário alterar o restante do código.

O que um método set pode fazer além de simplesmente atribuir um valor?

O método set pode:

Validar dados.

Impedir valores inválidos.

Converter formatos.

Executar regras de negócio antes de armazenar o valor.

Dê um exemplo prático de validação dentro de um set.

```
class Pessoa {  
    private $idade;  
  
    public function setIdade($idade) {  
        if ($idade > 0) {  
            $this->idade = $idade;  
        } else {
```

```
        echo "Idade inválida!";
    }
}
```

Nesse exemplo, a idade só será armazenada se for maior que zero.

3. Verdadeiro ou Falso — Encapsulamento

a) "Um atributo private pode ser lido diretamente fora da classe."

Falso (F)

Justificativa: atributos private só podem ser acessados pela própria classe. Uma tentativa de acesso externo gera erro.

b) "Getters e setters só fazem sentido quando os atributos são private."

Falso (F)

Justificativa: eles podem ser usados também com atributos protected, mas são mais comuns quando os atributos são private, pois ajudam a controlar o acesso aos dados.

c) "Encapsulamento aumenta a segurança e a manutenção do código."

Verdadeiro (V)

Justificativa: o encapsulamento protege os dados contra alterações indevidas e facilita a manutenção, pois centraliza o controle das informações na própria classe.

d) "É possível ter um getter sem ter um setter para o mesmo atributo."

Verdadeiro (V)

Justificativa: às vezes é desejável permitir apenas a leitura de um atributo, sem permitir sua alteração.

Exemplo:

```
class Produto {
    private $codigo;

    public function getCodigo() {
        return $this->codigo;
    }
}
```

Nesse caso, o código pode ser consultado, mas não modificado diretamente.

Resumo

Encapsulamento: protege os dados da classe.

private: acesso apenas dentro da classe.

protected: acesso na classe e nas subclasses.

public: acesso livre.

Getters: permitem leitura controlada.

Setters: permitem alteração controlada e validação dos dados.

4. Erro: acesso a atributo privado

O erro está nesta linha:

```
echo $p->nome;
```

O atributo \$nome é private, então não pode ser acessado diretamente fora da classe.

Correção:

```
<?php
```

```
class Produto {  
    private $nome;  
    private $preco;  
  
    public function __construct($nome, $preco) {  
        $this->nome = $nome;  
        $this->preco = $preco;  
    }  
  
    public function getNome() {  
        return $this->nome;  
    }  
}
```

```
$p = new Produto('Teclado', 150.00);  
echo $p->getNome();
```

```
?>
```

5. Erro: uso de variável sem \$this

O erro está nesta linha:

```
echo 'Aluno: ' . $nome . ' - Nota: ' . $nota;
```

Dentro da classe, os atributos devem ser acessados com \$this->.

Correção:

```
<?php
```

```

class Aluno {
    private $nome;
    private $nota;

    public function __construct($nome, $nota) {
        $this->nome = $nome;
        $this->nota = $nota;
    }

    public function exibir() {
        echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;
    }
}

$a = new Aluno('Carlos', 8.5);
$a->exibir();

```

?>

6. Erro: setter não bloqueia valor inválido

O erro está nesta linha:

```
$this->idade = $idade;
```

Mesmo quando a idade é inválida, o valor é atribuído.

Correção com else:

```
<?php
```

```

class Pessoa {
    private $idade;

    public function setIdade($idade) {
        if ($idade < 0) {
            echo 'Idade inválida!';
        } else {
            $this->idade = $idade;
        }
    }

    public function getIdade() {
        return $this->idade;
    }
}

```

```

$p = new Pessoa();
$p->setIdade(-5);
echo $p->getIdade();

```

?>

Também poderia ser corrigido usando return após a mensagem de erro.

7. Erro: instanciação sem new

O erro está nesta linha:

```
$c = Carro('Fusca');
```

Para criar um objeto em PHP, é obrigatório usar new.

Correção:

```
<?php
```

```
class Carro {  
    private $modelo;  
  
    public function __construct($modelo) {  
        $this->modelo = $modelo;  
    }  
  
    public function getModelo() {  
        return $this->modelo;  
    }  
}
```

```
$c = new Carro('Fusca');  
echo $c->getModelo();
```

?>

Resumo:

Use get para acessar atributos privados, \$this-> dentro da classe e new para criar objetos.